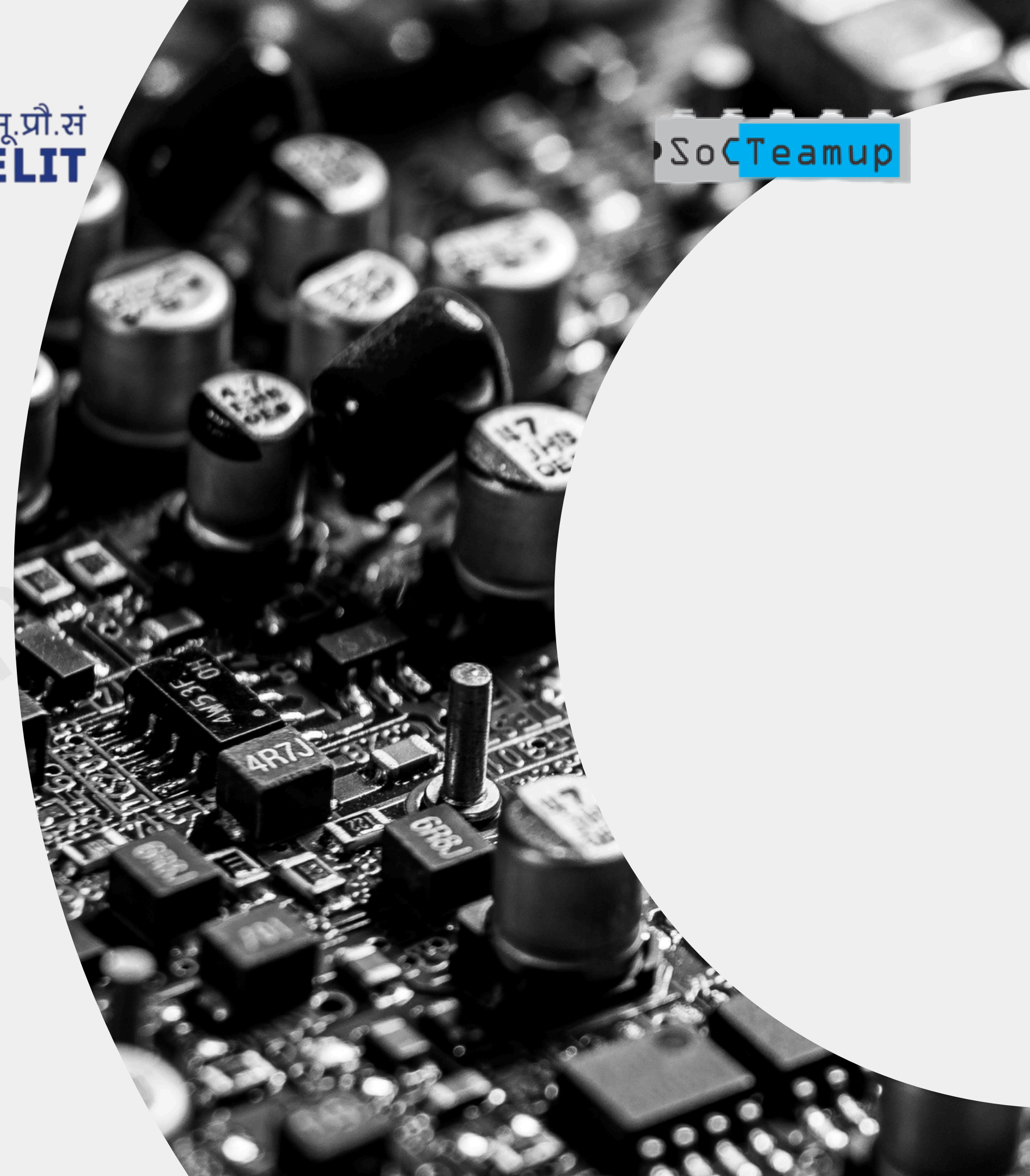


Key synthesis concepts parsing, elaboration, optimization phases



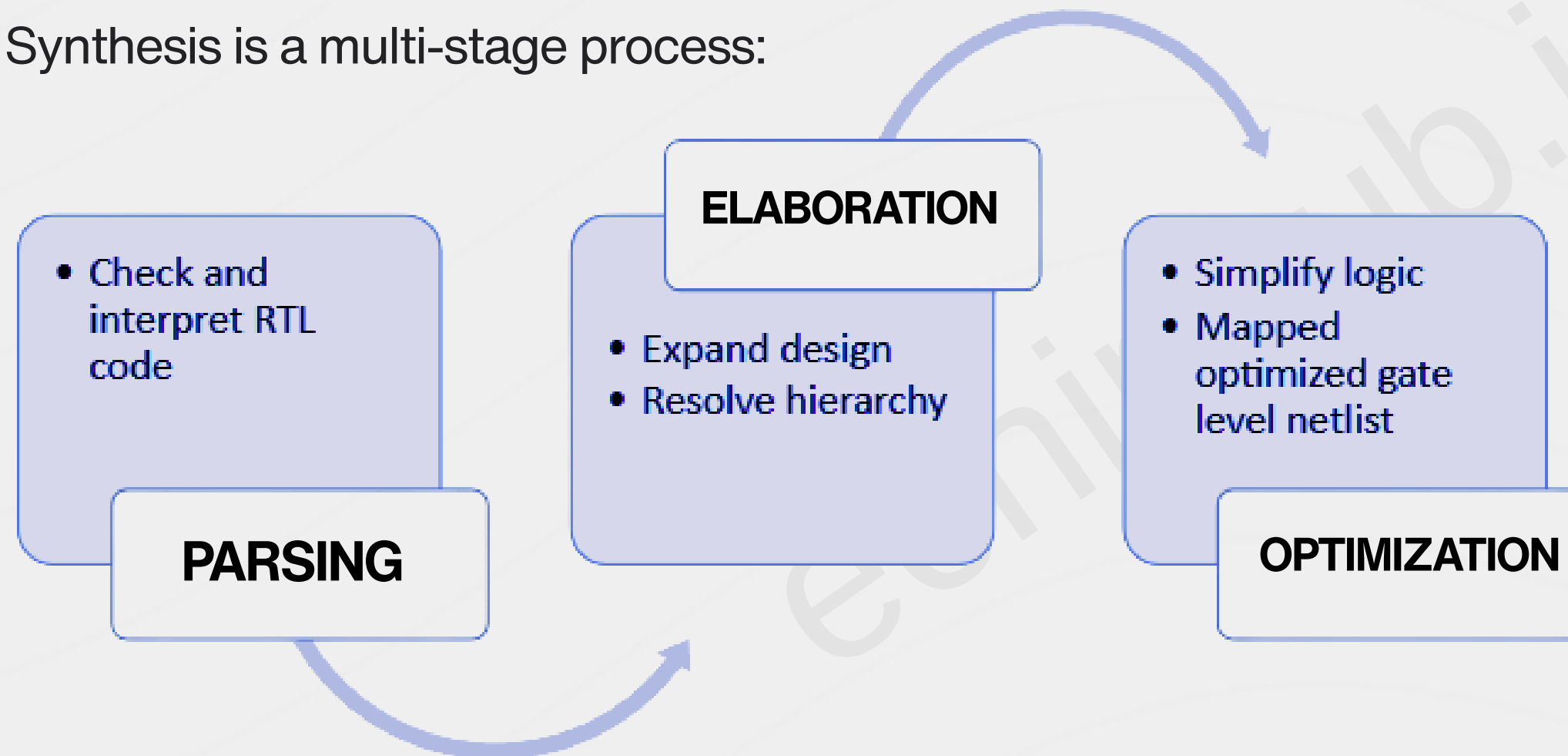


Outline

Introduction to synthesis tool flow
What is parsing?
Syntax and schematic analysis
Example: RTL parsing errors
What is elaboration?
Parameter and generate block resolution
Hierarchy and instance binding
Constant propagation
Register inference
Combinational logic optimization
Sequential logic optimization
Timing-driven optimization
Technology mapping recap
Final netlist generation
Summary and Best Practices

➔ Introduction to **Synthesis** Tool Flow

Synthesis is a multi-stage process:



Goal: Generate a clean and optimized gate-level netlist

➔ What is Parsing?

Parsing: It converts readable RTL into Yosys's internal format, catching syntax errors early before optimization passes like constant propagation.

CHECKS

- Syntax errors
- Signal declarations
- Module ports and datatypes

Tools use a parser to convert code into an internal model (AST - Abstract Syntax Tree)

Syntax and Schemantic Analysis

Syntax Check:

- Valid Verilog constructs

Schemantic Check:

- Are wires declared?
- Are ports connected properly?
- Are data types consistent?

Example:

- ✗ Missing “endmodule”
- ✗ Mismatched input/output port widths

Example – Parsing Errors in RTL

```
module bad_example(input a, output b)
    assign b = a + 1
endmodule
```

Parsing Stage flags faces issues:

- Missing semicolon
- No width declared
- Behavioral ambiguity in constant addition

What is **Elaboration**?

Elaboration = Design preparation

Elaboration turns parsed Verilog into a concrete circuit structure in Yosys.

- Expands module instances and connects hierarchy.
- Evaluates parameters, loops, and generate statements.
- Converts always blocks to gates and flip-flops

Think of it as compiling RTL for synthesis

➔ Parameter and Generate Resolution

Parameter Resolution

- Replaces parameters like `WIDTH=8` with actual numbers everywhere.
- Makes designs scalable by overriding values at module instantiation.

Generate Resolution

- Unrolls for-loops and if-statements into multiple real instances.
- Creates repetitive circuit blocks explicitly for optimization.

➔ **Hierarchy and Instance Binding**

Hierarchy and Instance Binding happen during synthesis elaboration after parameter/generate resolution.

Hierarchy

- Builds tree structure connecting parent modules to child instantiations (top-down organization).
- Preserves module boundaries initially for modularity before optional flattening.

Instance Binding

- Links each instantiation (e.g., `mux #(.WIDTH(16)) u1(...)`) to its module definition.
- Resolves port connections, creating wires between parent/child signals

Constant Propagation and Folding

Constant propagation and folding are RTLIL optimization passes in Yosys after elaboration.

Constant Propagation

- Scans RTLIL for wires/cells with known constant values and substitutes them in connected logic.
- Replaces parameters/signals (e.g., constant WIDTH=8) throughout the netlist.

Constant Folding

- Evaluates constant expressions (e.g., 3'b101 & 3'b111 → 3'b101) at compile time.
- Simplifies gates with constant inputs (constant 0 to AND becomes 0).

CONSTANT PROPAGATION

Replace static expressions with
values

CONSTANT FOLDING

Evaluate compile-time
expressions

Example:

assign $y = 1'b1 \ \& \ a$; $\rightarrow$ assign $y = a$;

Saves gates and improves performance

Register Inference

Register inference is how Yosys converts sequential always blocks into actual flip-flops during internal netlist creation.

- Detects clocked non-blocking assignments (e.g., `always_ff @(posedge clk) q <= d;`) and maps to DFF cells.
- Identifies edge-sensitive behavior from sensitivity lists and assignment patterns.
- Ensures simulation matches synthesized hardware timing.

➔ Combinational Logic Optimization

Combinational optimization simplifies gate-level logic by removing redundancies and factoring expressions during synthesis.

- Constant simplification turns $out = a \& 1'b0$ into $out = 1'b0$, removing useless gates.
- Redundant removal changes $out = a \mid \sim a$ to $out = 1'b1$ since it's always true.
- Boolean factoring simplifies $y = (a\&b) \mid (a\&\sim b)$ to $y = a$ by merging common terms.
- Gate merging cancels $\sim(\sim x)$ back to just x .

Sequential Logic Optimization

Sequential optimization balances flip-flop paths, eliminates unnecessary registers, and improves clock domain timing.

- Register merging combines two back-to-back flip-flops into a single flip-flop.
- Retiming moves flip-flops across combinational logic to balance critical paths.
- Clock gating disables the clock signal to idle registers, saving dynamic power.
- Redundant FF removal eliminates flip-flops that only drive constant values.

Timing-Driven Optimization

Timing driven optimization uses timing analysis during synthesis to target critical paths and meet clock constraints.

- Identifies longest paths (critical paths) using estimated gate/wire delays from Liberty libraries.
- Applies aggressive optimizations like gate sizing or buffer insertion only where timing slack is negative.
- Iteratively re-analyzes after each transformation until all paths meet setup/hold requirements.
- Balances timing improvements with area/power by focusing effort on violating paths only.

➔ Technology Mapping Recap

- Final logic mapped to standard cells from .lib
- Each gate has known area and delay



Final output:

- Synthesized Verilog netlist
- .sdf (delay format)
- Reports (area, timing, power)

Summary and Best Practices

Parsing checks RTL syntax and meaning
Elaboration builds a resolved hardware graph
Optimization improves area, timing, and power
Constant propagation and register inference are key steps
Use clean RTL and check logs for warnings
Understanding internal flow for Writing better RTL

Thank You

echiphu

